

Pick only the required commit and squash other commits with the one picked.

The changes can now be submitted to the maintainer for incorporation into the official repository.

```
$ git format-patch origin (or) $ git format-patch -1
```

The above command will generate files of the type `0001-xxxxx.patch`. If the commit squash was done, only one file will be generated. Send these files to the maintainer via email.

What maintainers normally expect is that the mail containing a patch to be incorporated in the official repository should also mention the branch to which the patch is to be applied, the bug-ID, and the proof of review approval (such as the review URL). The maintainer will usually send a confirmation once the patch is applied.

You can now pull changes from the main repository into your *master* branch and delete the *working* branch, using the following code:

```
$ git checkout master
$ git pull --all
/* Verify submitted changes are present in "master" */
$ git branch -d <branch-name>
```

Working on multiple work items

Normally, a developer works on multiple work items. For example, the developer may be assigned a feature and multiple bugs. Having a separate cloned repository for each work item is ideal. But due to disk quota, this is a luxury for most developers. Instead, they need to work on the same cloned repository. This is facilitated by Git, which provides a way to stash work and come back to it later.

As mentioned earlier, each work item must be on a separate branch. So, we can save uncommitted code before switching branches by stashing the changes, as shown below:

```
$ git stash save "<message>"
$ git checkout <other-branch>
-
$ git checkout <earlier-branch>
$ git stash pop --index
```

You may stash changes in multiple branches. To view the stashed changes, use the following code:

```
$ git stash list
```

To apply a specific stash once you are back in a branch, use the command shown below:

```
$ git stash apply <stash-name> --index
$ git stash drop <stash-name>
```

Note that *git stash drop* is used to remove a stashed set of changes after it is applied via *git stash apply*. Git does not remove it automatically, on the assumption that you may need it later for some other branch. This behaviour is unlike that of *git stash pop*, which removes the stashed changes that were popped out.

Labelling

Any project would want to label code at specific milestones. Git allows us to label any specific code snapshot. Apply a label with a comment, using the following code:

```
$ git tag -a <label> -m "<info on label>"
```

To get to a labelled code snapshot, use the following command:

```
$ git checkout <label>
```

Release management

Release management deals with creating a release version of your project for external consumption. Project members label the code snapshot used to create the release version. This helps the project team to go back to the specific code snapshot at any time in the future, often to resolve issues reported in the released product.

The release is usually done from the official repository, where you need to go through the sequence of steps that follow:

- Apply a label with *ver#* as per project conventions, e.g., *myproject_release_v1.0*
- Also, create a branch with *ver#* as per project conventions, e.g., *myproject_release_1.0_branch*
- There are some rules when it comes to *release* branches in the official repository:
 - *Release* branches must never be deleted.
 - *Release* branches must not be rebased with the *master* or any other branch. *Release* branches are long-lived branches, unlike *development* (*working*) branches described earlier.
 - Label any hot-fix releases done out of the *release* branch as per project conventions.

Cherry-picking

When defects are reported in a released version, there could be a need to fix code in a *release* branch and issue updates.

The recommended workflow for handling issues in a *release* branch is as follows:

Step 1: Check if a defect exists in the *master* branch, and if so, fix it in the *master* branch.

Step 2: Cherry-pick the commit into the concerned *release* branch by using the following code:

```
/* Do bug fix and commit to master */
$ git checkout <release-branch>
```

Continued to page 85...